Fine-grained role-based access control by encryption

Andreas Severin Hauch Trøstrup
Computer Science
IT University of Copenhagen
atro@itu.dk

Lucas Frey Torres Hanson
Computer Science
IT University of Copenhagen
luha@itu.dk

***Abstract*—**

I. Introduction

Today, when you want to use a data lake, you have two options. *Option 1:* Select a "fully managed all-in-one cloud platform"[1] that integrates with data lakes and provides fine-grained access control. *Option 2:* Provide the external query engines the ability to interact with the data lake, but lose the ability for fine-grained access control. So either you choose *Option 2* and limit who the data can be shared with, based on whether they should only be able to access part of the data, or choose *Option 1* and limit the query engines the data scientists want to use.

The Object Stores, such as Amazon S3, being the data lakes, do not understand a granularity finer than file level. This limitation prevents external access control services from governing access to a finer granularity, such as column-based. If, instead, the mechanism for access control was moved within the data lake itself, this could enable fine-grained access control. In the paper "Reviewing options for fine-grained Role-Based Access Control in Data Lakes", by Trøstrup and Lucas[1], the solution called "Object Store Wrapper Service" (OSWS) was proposed.

The idea of OSWS is to encrypt the Parquet files within the data lake using Parquet Modular Encryption (PME), which supports encrypting each column. The keys will be stored in a key vault and in wrapped format within the Parquet files, and will be mapped to roles, to limit which roles have access to specific columns. OSWS will expose an S3-compatible API, which can then decrypt the data for the "fully managed all-in-one cloud platforms" and the external query engines when getting normal S3 requests. Thus, regardless of where to read the data from, it will have been ensured that they can only see what they are supposed to.

In this thesis, an MVP of OSWS will be created to prove that a system supporting third-party query engines, such as DuckDB, without modifications, can use OSWS, though, as quickly realized, interoperability with the "fully managed all-in-one cloud platforms" is partly a job that the respective companies have to allow[2]. After developing the MVP and coupling a query engine to it, experiments will be conducted to evaluate the performance and security of the system, compared to available solutions. Metrics include end-to-end read and write latency, as well as the cryptographic overhead, in relation to the size of the Parquet file, introduced by the OSWS. Experiments will use data sizes representative of realistic data sizes.

A. Research Questions

Is it possible to create a wrapper service that ensures access control on column-level using RBAC and has interoperability with existing query engines and "fully managed all-in-one cloud platforms" without modifying them?

B. Objectives

This research paper will have the following objectives:

1) Create OSWS MVP using an underlying S3-compatible Object Store
2) Ensure OSWS enforces access control using RBAC outward using encryption
3) Make it compatible with "fully managed all-in-one cloud platforms"
4) Make it compatible with query engines
5) Benchmark OSWS to measure the latency it adds.
6) Reflect on design decisions and which alternatives would have been better

II. Related Work

A. Reviewing options for fine-grained Role-Based Access Control in Data Lakes

This thesis is a continuation of the research paper titled "Reviewing options for fine-grained Role-Based Access Control in Data Lakes"[1].

This was the foundational work to reflect on how OSWS would theoretically work. After implementing the OSWS, discoveries revealed that it did not work as initially planned and as described in the research paper.

The biggest change from the research paper is that OSWS does not support third-party query engines being able to decrypt locally, but has to get the decrypted Parquet through OSWS. As also mentioned later, "fully managed all-in-one cloud platforms", such as Snowflake or Databricks Unity Catalog, are not guaranteed to be supported as they would have to whitelist a URL on which OSWS runs, and have thus not been able to be tested; it is not certain that, given a whitelist, it would work. Lastly, data lake-specific systems, such as DuckLake, are not supported either, see more in Section VII,

[1]See Section III.L
[2]See Section VII

due to modified Parquet files. Though this is an issue that could be removed in the future, it will require OSWS to add another logical layer that bridges the two design choices.

B. Membrane: A Cryptographic Access Control System for Data Lakes

Membrane, referenced in Trøstrup and Lucas[1] as well, is a proposal for ensuring FGAC at the cell-level and encryption-at-rest in data lakes with a custom format, Kumar et al.[2], thus resembling what OSWS sets out to do. Besides making the original client the one responsible for handling the original encryption keys, to provide access to others, it also makes the query engine have to download the whole file to then decrypt it with a provided view, on its machine. The advantage of this is that once the query engine has the data, given the view, they only have to make computations on the relevant data. This differs from OSWS, where it actually runs computations on all data, but instead only gives the query engine the finalized data. The computation volume for Membrane is therefore correlated to the size of the view size, whereas for OSWS, it is related to the file size, but OSWS supports existing query engines which can interact with S3.

Missing comparison of if there are diff between research project OSWS and current OSWS against Membrane

C. Uber Paper

We should probably cite the uber paper

III. Background

Data lakes are not a new concept, but the term was first coined in 2011 by Dixon[3], and some of the terminology is a bit inconsistent. This section will describe the terminology used within this thesis to ensure consistency. A lot of the definitions will be taken directly from the previously written research paper, which will be referenced and quoted accordingly.

A. OSWS

Object Store Wrapper Service (OSWS) is the concept of a layer on top of an Object Store and which support S3-compatible API endpoints. It is meant to provide fine-grained role-based access control by encryption. This should be achieved through using PME, see Section III.K, and will only decrypt the columns which are authorized to the client requesting a given Parquet file. See more in Section IV.

B. JIT Provisioning

Just-In-Time (JIT) Provisioning is an identity management process handling creating user accounts when they are needed. [4] This is needed during the initial user creation, see Section IV.D.b.

C. RBAC

In Trøstrup and Lucas[1], Role-Based Access Control (RBAC) was defined as follows:

> Role-based Access Control (RBAC) [5] is an approach to access control based around assigning permissions to roles, and grouping users into these roles. Access control is then managed by controlling what roles are allowed to do what, instead of managing each user individually. The "Core RBAC" model, as defined by Ferraiolo et. al. 2001, [6], consists of five basic elements: *users*, *roles*, *permissions*, *operations* and *objects*. A *user* is some actor, typically a human user, but could also be an autonomous actor, who needs to be authorized for some set of actions. A user is assigned one or more *roles*. A role is simply some named collection of authorizations; for example, a job title. A role is assigned one or more *permissions*, which are what give authorization to perform an *operation* on a secured *object*. […] In addition, Core RBAC also describes *sessions*. A *session* is a mapping of one user to a subset of the roles they are authorized for; i.e. a user "activates" one or more roles that have the necessary permissions, when they need to perform a specific operation on an object. This allows for finer control of roles that are active in a given context, to prevent unnecessary access levels. Commonly, a hierarchy is introduced to allow for structuring roles which should inherit some base permissions; for example, a Doctor and Nurse role might inherit permissions from some base "Healthcare Worker" role. This is referred to as "Hierarchical RBAC" [6].
>
> — Trøstrup and Lucas[1]

D. OIDC

OpenID Connect (OIDC) is an identity authentication protocol based on the authorization OAuth 2.0 framework.[7] It provides developers with a standardized, simple way to verify the identity of users trying to access web applications. It allows users to authenticate using their existing accounts previously registered with an OpenID Provider, such as Microsoft Entra ID[8] or PocketID[9], which the web application can then contact to verify their identity. Thus, it eliminates the need to implement an authentication layer in the web application.

E. KMS/KV

Key Management Service (KMS) and Key Vault (KV) are both services used to manage keys.[10], [11] KMS is widely used within data storage, and is what AWS use to refer to their system, whereas Azure uses KV. This paper will use the term KV, as OSWS has been set up against Azure KV but could have used KMS, see more in Section IV.B.c. KV is a way to store cryptographic keys. When the cryptographic keys are within the KV, they can no longer be retrieved, and thus if data has to be encrypted/decrypted, it either has to be sent to the KV or envelope encryption can be used, so the KV is responsible for unwrapping the DEK, see more in Section III.I.

F. Data Lake

In the previously written research paper, data lakes were defined as follows:

> A "data lake" is a centralized repository of data, which could be structured, semi-structured or unstructured.[3] It differs from a traditional database in that a data lake separates compute and storage layers. The data itself is stored in object stores like Amazon S3[12], in file formats like CSV, Parquet[3], etc. The compute nodes are simply any query engine, such as Apache Spark [13], that connects to the object store and uses the data. This could be a data scientist working on their own machine, who connects to a data lake and runs queries; or it could be a part of a managed all-in-one platform like Snowflake, see more in Section III.L. As the object store just contains raw data, both structured and unstructured, this can make it more complicated to query, especially for large data lakes. One approach to querying is "schema on read", in which the query engine infers a schema at query time based on the files in the data lake. This is in contrast to "schema-on-write" used in traditional relational databases, where the schema is strictly enforced when writing. This doesn't work well for data lakes due to the nature of storing raw data in object stores. Typically, data lakes introduce a *catalogue* on top of the object store that manages file schemas, metadata and snapshots, allowing for structured reads and more performant querying through techniques like partitioning, i.e. filtering unnecessary data. This has also been referred to as a *data lakehouse* .[14]
>
> — [1]

G. Parquet

In the previously written research paper, Parquet files were defined as follows:

> Parquet [15] is a columnar file format supported by many data processing systems.
>
> A Parquet file is structured using row groups, which contain columns, along with a footer which contains metadata. The columns are structured as column chunks, which contain several *data pages*. A data page contains a header, which describes information like the size of the page, and following that, the actual values as bytes. The file footer contains metadata that specifies what is stored in the file and where, that is, where a reader should look to find a specific column, or the statistics of a column (e.g. min/max). Appendix 1.a shows a visual representation of a Parquet file that also uses encryption, which will be described later. The left side of the figure shows row groups, columns and data pages, along with their headers. The right side shows the footer of the file, with the file metadata containing metadata on each row group, which in turn contains metadata on the column chunks in that row group. The arrows link information about where to read, such as offsets, to where they point the reader. The objects that hold metadata, like statistics, and structural information, including physical offsets, such as the footer and page headers, are serialized using Apache Thrift[16], a language-agnostic serialization framework, and referred to as "Thrift Structs". Essentially, the object that contains the information, for example, a `PageHeader` object (on the left side of Appendix 1.a) or a `ColumnMetaData` object (on the right side of Appendix 1.a), is serialized into bytes using Thrift, which can then later be restored to the same object.[17]) Then, to read a Parquet file, the reader de-serializes the Thrift structures to get the necessary metadata, and then reads the actual data values using the information from the metadata.
>
> — [1]

H. TTL

Time-to-live (TTL) is a way to define the lifetime of some item within a dataset. When either the item's TTL expires or the dataset is full, the item closest to its TTL is removed.[18] In OSWS, it is a way to define how long the lifetime of the encrypted Parquet files and the unwrapped DEKs, see Section III.I, can live in storage and memory. When the TTL is reached, the data will be removed and has to be refetched from either S3 or the KV when needed. OSWS defines two different TTLs as a way to follow general security guidelines, which can be read about in "NIST Special Publication 800-57 Revision 5 Recommendation for Key Management"[19], where OSWS uses a TTL of five minutes for DEKs authorized to admins, and the rest have a TTL of 15 minutes.

I. Envelope Encryption (KEK, DEK)

As KV, see Section III.E, does not support retrieving keys, OSWS uses envelope encryption. This means that during encryption of a column, cryptographic keys for each are generated, called a data-encryption-key (DEK), which is used to encrypt the columns. Then, for the single Parquet file, a single key-encryption-key is generated, resulting in less data compared to if each DEK had its own separate KEK within KV. This KEK then encrypts all the DEKs, also called wrapped DEKs, and gets stored in KV. The wrapped DEKs are then stored in the metadata for the Parquet file together with a KEK id.
When the columns are to be decrypted, the wrapped DEKs are sent to KV to get unwrapped, and the columns can be decrypted.[20]

J. Trust Boundary

Skal vi bare nakke? Vi referer til Trust Boundary senere i opgaven A trust boundary is a boundary where the level of trust is checked. So the place where information from one side of the boundary is validated, and if it is valid, it will permeate to the other side, based on the restrictions. This can be both if users should be able to access the data on the

[3] Section III.G defines Parquet

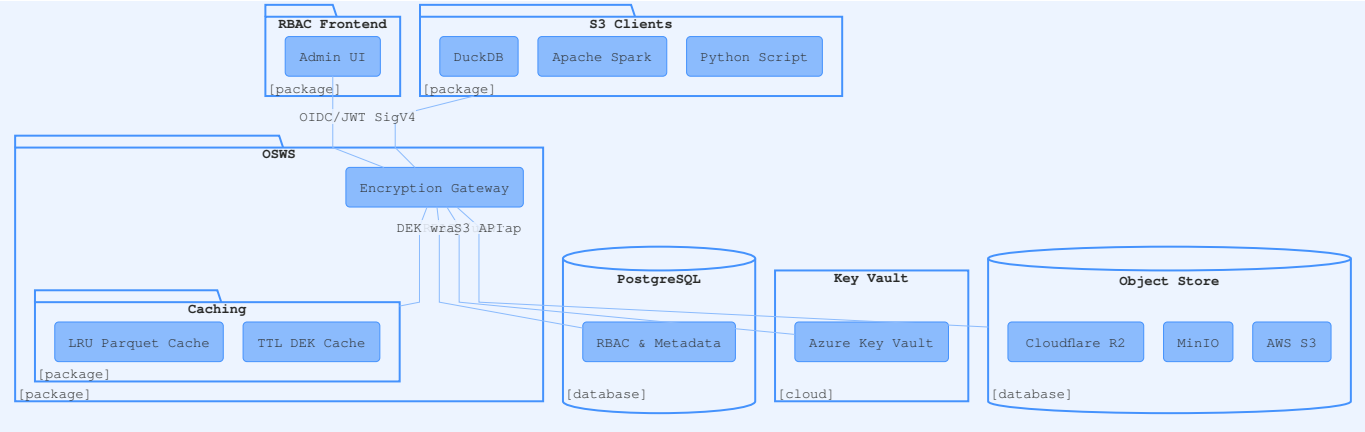

Figure 1. Visual overview of how the OSWS project is structured together with the client entry-point into OSWS

other side or if data should be able to be stored on the other side.[21]

K. PME

Parquet Modular Encryption (PME) is a way to allow granular control of how the Parquet file data and metadata should be encrypted. Singular columns can be encrypted, and the footer as well. For OSWS, footer encryption is not enabled. There are multiple reasons for this, but they are argued in **ref sec**. When encryption is used, serialized Thrift structs are encrypted using the given cryptographic key, and the data pages themselves are encrypted as well. In OSWS, it is saved as a tuple in the footer, containing a reference to the KEK in KV and the wrapped DEK. This allows OSWS to get the DEK unwrapped to then decrypt the column **ref sec**.[17]

L. Fully managed all-in-one cloud platforms

In this paper, "Fully managed all-in-one cloud platforms" will be used to define solutions such as Snowflake and Databricks. Access control within those solutions is managed by the query engines together with the catalogue. This means the query engine is trusted to filter out rows that are not permitted, and it then masks or removes those values, which is possible as the platforms themselves manage it.

M. Vended Credentials

"Vended credentials" is a way to manage temporary access to files within S3, but the issue is that it can only gate access on the file level, based on limitations in S3. This is the way Lakekeeper and Apache Polaris currently manage their access grants.[22], [23]

IV. System Design

OSWS set out to provide columnar access control to 3rd party query engines and "fully managed all-in-one cloud platforms" by encrypting columns individually using PME, provided through the Parquet Sharp library, see Section IV.E.b. But due to how some of these work on specific query engines can work with OSWS, see Section VII for more info. Though some changes can be made to bridge these two different design choices, see more in Section VII. What the current OSWS solves is that query engines, which read metadata from object storage and do not use cached values, will have columnar access control enforced on the data they read. This ensures that, from the query engine's perspective, they are using S3 directly, but in reality use OSWS, and they can now read and write data to S3 as usual, but role-based access control is ensured on the column level. So if you have a dataset with columns `name`, `birthday`, and `relational status`, and the query engine is not granted access to `birthday`, it will receive, depending on the setup of OSWS, all columns, but `birthday` will be either null values or an encrypted column.

Below, the design choices will be described in more depth, together with deeper technical information.

A. Architecture Overview

OSWS is set up as a wrapper for S3 and thereby provides S3-compatible API endpoints for clients. The purpose of OSWS is to provide column-level access controls to S3, whilst not modifying the S3 API. This ensures that most S3-compatible query engines are able to use OSWS without modifications and that they will only be able to read what they are supposed to.

The system is built on ASP.NET Core 10, and uses: PME, minimal API, PostgreSQL for RBAC, and was manually run and configured with Azure Key Vault for key management, Cloudflare R2 as Object Store (R2 is S3 compatible).[4] Minimal API being that there are no controllers, and that minimal dependencies exist to set it up, Anderson and Dykstra[24].

B. Layered Architecture

Address comment: (6) The headline says "Layered Architecture", but you describe the "repo" in the text and in Figure 1. It would be better if you described the architecture. So instead of describing which source files exist in which directories, describe the main components of your system, what they do, and how they interact with each other. As Figure 1 include an architectural diagram, which normally consists of system components, system boundaries, and input and output. Try to find literature

[4]Codebase available at github.com/lucasfth/osws

that describes the architecture of a system or database and modify your description to follow more common standards. The repo is structured into six main .NET projects and one frontend project, written in React. You can see the structure visually in Figure 1 together with the client entry-point to OSWS.

a) *OSWS.WebAPI:*

WebAPI is the entry-point that, from the query engine's perspective, is S3. It is responsible for hosting the ASP.NET Core minimal API and registers all the services.

- Files within `Authentication` are responsible for the authentication schemes which are provided in incoming requests, including AWS Signature V4 (SigV4) for S3 API calls, and JWT Bearer validation for OIDC-authenticated calls.
- Files within `Endpoints` are all the endpoints OSWS supports, both for S3-compatible API endpoints and the endpoints needed to use the frontend to ensure RBAC can be handled.
- Files within `Extensions` ensures to load up all configurations for the setup of OSWS. This includes OIDC, rate limiting, endpoints, and object store to use for this specific setup of OSWS.
- `Interfaces` folder contains the S3-endpoints interfaces, so implementations can easily be changed out.
- `Models` contain `OidcUserInfo`, which defines the record of the type `OidcUserInfo`.
- `Services` defines the services used to handle and resolve users, Parquet uploads, which columns a user is authorized to see, transitive roles memberships, S3 object retrieval, and user info retrieval.
- `Program` is the entry point to run OSWS itself and add all relevant services.

b) *OSWS.ParquetSolver:*

ParquetSolver handles the cryptographic operations for the Parquet files.

- `Interfaces` contain the interfaces `IDekCache`, `IParquetReader`, and `IParquetWriter`.
- `KeyRetriever`: Implements `ParquetSharp.DecryptionKeyRetriever`, and handles retrieving the correct keys given the metadata within the Parquet files.
- `ParquetWriter`: Implements `Interfaces/IParquetWriter` and takes a plain-text Parquet file and encrypts the columns if needed. This is done by generating an AES key of a size defined either within `OSWS.Common/Configuration/EncryptionSettings` or in `appsettings.json`. The DEK is then wrapped in a KEK (see Section IV.B.c).´ The wrapped DEKs are serialized, and the Parquet file is written with `ParquetSharp` using the serialized DEKs in the footer, and it is then sent to S3.
- `ParquetReader`: Implements `Interfaces/IParquetReader`. It takes the encrypted Parquet file and copies it to a decrypted version, decrypting using the metadata stored in the file. Forbidden columns are masked with dummy data.

Parquet Sharp was chosen as it supported PME, but it does not support partial decryption/reads of Parquet files, as it has to copy over all the columns to read. This results in the fact that, from OSWS to S3, ranged requests are not supported, but from a client's perspective, it is supported (a solution to this has been proposed in Section VII.B).

c) *OSWS.KeyManager:*

KeyManager provides an abstraction over cryptographic key storage and the relational model.

- `AzureKeyVaultProvider`: Implements `IKeyVaultProvider`. It creates keys (which are defined in `OSWS.Common/Configuration/EncryptionSettings`) in Azure Key Vault, performs the wrap and unwrap of the DEKs server-side, using the algorithm defined within `OSWS.ParquetSolver/Helpers/Cryptography`.
- `InternalKeyVaultProvider`: Implements `IKeyVaultProvider`. It works the same as `AzureKeyVaultProvider` but runs on OSWS itself.

  As long as `IKeyVaultProvider` is implemented together with a key vault provider, which means it supports the following functions: encrypt, decrypt, get key info, and list keys, most providers should be able to be used.
- `OswsContext`: Implements `DbContext`. Defines the relational schema over PostgreSQL, including users, roles, role assignments, role inheritance, permissions, columns, keys, external identities, and S3 credentials.

d) *Shared Libraries:*

- `OSWS.Models` defines all the DTOs and entities used within the solution.
- `OSWS.Common` contain classes containing settings for the project, including. `EncryptionSettings`, `CacheSettings`, `S3Settings`, `KeyVaultSettings`, and `RateLimitSettings`. They are bound to `appsettings.json` at startup, and they also include validation logic for the configuration.
- `OSWS.Library` has utility helpers for AWS credential normalization, S3 metadata translation, HTTP range request parsing, parameter validation, and XML extensions.

C. Database Design

An Entity Relationship describing the database design can be seen in Figure 2. The database is mostly used for RBAC metadata; however, it also stores credentials used for AWS Signature V4 request signing, see Section IV.D.a, and external identity information from the OpenID Provider(s).

D. Authentication

Due to S3 requiring signage, the calls from the clients going to OSWS are signed as well, and as a result, OSWS cannot reuse the signature and encrypt the Parquet columns. Internal authentications were therefore needed, and OSWS then has its own signature for S3.

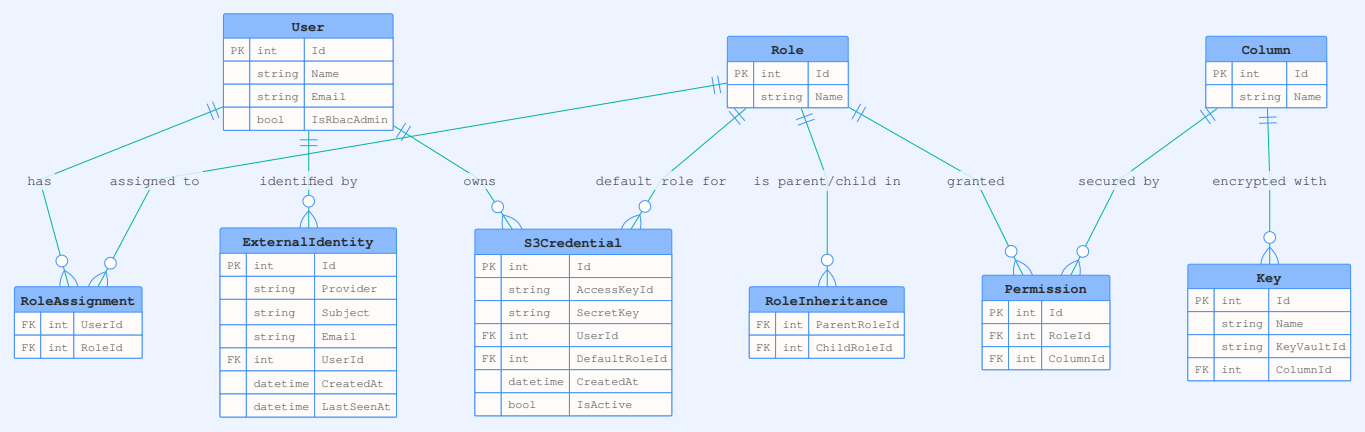

Figure 2. Entity Relationship Diagram showing the design of the database

a) *AWS Signature V4 (S3 API):* Because OSWS had to use an S3-compatible API, it also had to use the same validation as S3. A custom `SigV4AuthenticationHandler` has been created to parse the authorization header and extract the `AccessKeyId`. The keys corresponding to the `S3Credential` record are then looked up in the internal PostgreSQL, see the `S3Credential` table in Figure 2. The signature is verified using the stored secret key. On success, a `ClaimsPrincipal` is created carrying the user's database ID, name, email, and default role.

This follows the AWS specification "Authenticating Requests (AWS Signature Version 4)"[25].

b) *OIDC:*

The React frontend, for handling RBAC, uses OIDC for authentication flow. The OIDC provider chosen for OSWS is Pocket ID, as it is simple to set up, is open-source, and uses passkeys, which are more secure than standard MFA, as "common multifactor authentication methods can be intercepted or relayed"[26].

The backend still supports other OIDC providers. To use other providers, `OSWS.WebApi/appsettings.json` needs to be updated to reflect the change, and the frontend `.env` has to point to the specified authority and client ID.

On the first login, the `api/me` endpoint triggers JIT provisioning, and a new `User` record and `ExternalIdentity` record are created in the database. Subsequent logins will synchronize the OIDC provider's claims.

E. Envelope Encryption

To encrypt the Parquet columns, OSWS needs to use DEKs and KEKs to achieve envelope encryption[27]. This deviates from the original proposed solution in [1], as KV does not support key retrieval, and the overhead of sending a Parquet column to KV each time for decryption and encryption is high. By using envelope encryption, OSWS can cache the unwrapped DEKs with a specified TTL to allow quicker decryption. This results in a solution with little overhead, more about this in Section VI.B.a, and ensuring column-level access control.

a) *Key Hierarchy:*

The KEK sizes are specified within the KV that the admin chooses. Due to having student credits available in Azure, which was only available for a low-cost KV, RSA-2048 was used. The KEKs are created in OSWS (inside `OSWS.KeyManager/Providers/AzureKeyVaultProvider`), but after that, the KEK never leaves KV again, and are then called by OSWS to wrap and unwrap the DEKs.

For the DEKs OSWS, create these themselves. These symmetric AES keys have sizes 128, 192, or 256, and are created during the encryption of Parquet files.

b) *Encryption Flow:*

The encryption flow works as follows:

1) Client uploads unencrypted Parquet file, via `PUT /s3/{bucket}/{key}`.
2) OSWS creates an RSA-2048 key and is tagged with the uploading user's role.
3) For each column designated for encryption, an AES DEK of specified sizes is generated and encrypted, within `OSWS.ParquetSolver/Helpers/Cryptography`.
4) Each DEK are then wrapped by using the KV, then serialized, and put into the footer of the Parquet file.
5) The encrypted Parquet file is then written using Parquet Sharp.
6) Parquet file is then sent to the S3-compatible object store.
7) Columns, key IDs, and permissions are persisted in local PostgreSQL.

c) *Decryption Flow:*

The decryption flow works as follows:

1) The client requests a Parquet file, via `GET /s3/{bucket}/{key}`.
2) OSWS fetches the encrypted Parquet file, first tries in local cache, then if not found, it goes to S3-compatible object store (see `OSWS.WebApi/Services/Services/S3ObjectFetcher`)
3) Wrapped DEKs are read from the Parquet footer.
4) For each wrapped DEK, OSWS checks the in-memory DEK cache. On a cache miss, it calls the KV decrypt method to unwrap the DEK and caches the result.
5) The user's effective roles are computed via Listing 1.

6) The permitted columns are now decrypted, while the others are replaced by dummy columns[5]
7) The decrypted Parquet stream is returned to the client.

F. RBAC

In Section III.C, RBAC is defined to consist of: *users*, *roles*, *permissions*, *operations*, *objects* and *sessions*. Our implementation of RBAC follows this mostly, but differs in a few ways. In Figure 2, the core RBAC entities are shown: the tables `User`, `Role`, `RoleAssignment`, `RoleInheritance`, `Column`, and `Permission`. `User` and `Role` have a many-to-many relationship through `RoleAssignment`. Likewise, `Role` and `Column` have a many-to-many relationship through `Permission`. The `Column` table can be seen as the *object* of Core RBAC, as it is currently the only secured object to which access is controlled. There is however no concept of *operations*, only whether or not access is granted. There is also no formal concept of *sessions* as described by D. F. Ferraiolo and R. Kuhn [5], however, when a user is authenticated through their S3 Credential, their assigned roles are loaded. In this way, it can be seen as an activation of all the users assigned roles.

a) *Role Hierarchy:*

OSWS also supports Hierarchical RBAC through the `RoleInheritance` tables. This is a self-referential table in which a hierarchical relation is created through the `ParentRoleId` and `ChildRoleId` foreign keys, so that a *parent* inherits from a `child` (see `RoleInheritance` in Figure 2.) This means that all permissions of *child* also become permissions of *parent*. To get the full set of effective roles for a user, a recursive SQL query is used to navigate the hierarchy. This can be seen in Listing 1.

```
WITH RECURSIVE effective AS (
  SELECT ra."RoleId" AS "Id"
  FROM "RoleAssignments" ra
  WHERE ra."UserId" = {userId}
  UNION
  SELECT ri."ChildRoleId"
  FROM "RoleInheritances" ri
  JOIN effective e
  ON ri."ParentRoleId" = e."Id"
)
SELECT DISTINCT r."Id", r."Name"
FROM "Roles" r
JOIN effective e ON r."Id" = e."Id"
```

Listing 1. A recursive SQL query to get the full set of effective roles given a `userId`.

b) *Column-Level Permissions:*

Each *Permission* record maps a *Role* to a *Column*. This means that a column is decrypted if and only if one of its roles grants access to the given column. This ensures fine-grained access control. If a Parquet file *P1* with columns *name*, *age*, and *civil_personal_register* exists. Then a user *U1* and a user *U2* can respectively give access to role *R1* and *R2*. *R1* provides access to only *name* and *R2* only to *age* and *civil_personal_register*. Then *U1* would see a table full of valid *names* whilst *age* and *civil_personal_register* are either null values or some dummy encrypted values (depends on the configuration of OSWS), while *U2* would see *age* and *civil_personal_register* values.

[5]Due to limitations in Parquet Sharp, it is not possible to leave the encrypted column alone, and thus has to be replaced.

G. Caching

There are two tiers to the caching system used within OSWS with the intention of minimizing latency from remote services.

a) *Encrypted File Cache:*

Given the configuration of OSWS, the Parquet files fetched from the object store can be cached. It uses the LRU (Least Recently Used) policy to cache the encrypted Parquet files on the local file system. They are keyed with `SHA256(bucket::key)`, see `OSWS.ParquetSolver/Helpers/EncryptedFileCache.cs`. As the files are stored in their encrypted format, no new trust boundary is introduced.

b) *DEK Cache:*

Unwrapped DEKs are cached in-memory to avoid repeated calls to KV, as it is one of the major latency attributes, see Section VI.B. The cached DEK is keyed with the KEK identifier, and in the configuration, the TTL can be defined for regular and admin users, as admin users' keys are most often more privileged. The cache enforces a maximum capacity and evicts the expired entries first and then the oldest entries by expiration date.

H. API

OSWS exposes three different endpoint groups.

a) *S3-Compatible Endpoints:*

A subset of S3 required endpoints is implemented to allow for the object store operations. These operations can be seen in Table I.

TABLE I
S3-Compatible endpoints provided by OSWS

Method	Path	Description
GET	`/`	List buckets
GET	`/{bucket}`	List objects in a bucket
PUT	`/{bucket}`	Create bucket
DELETE	`/{bucket}`	Delete bucket
POST	`/{bucket}?delete`	Delete multiple objects in bucket
HEAD	`/{bucket}`	Retrieve bucket metadata
HEAD	`/{bucket}/{key}`	Retrieve object metadata
GET	`/bucket}/{*key}`	Retrieve an object (with decryption and column filtering for Parquet)
PUT	`/{bucket}/{*key}`	Store an object (with encryption for Parquet)
DELETE	`/{bucket}/{*key}`	Delete an object

Currently, non-Parquet files pass through the encryption step, and the endpoints defined are only to allow operations made by most query engines, and thus suffice for making

an MVP. In the future, this should be extended to also allow non-Parquet files to be encrypted and then include the KEK reference within the Parquet file referencing it, and the endpoints should also be extended.

b) *Application Endpoints:*

OIDC-protected endpoints for the web frontend. See the endpoints in Table II.

TABLE II
APPLICATION ENDPOINTS PROVIDED BY OSWS FOR WEB FRONTEND

Method	Path	Description
GET	`/api/me`	Current user profile (triggers JIT provisioning)
GET	`/api/credentials`	List the user's S3 credentials
POST	`/api/credentials`	Create a new S3 credential (access key + secret key)
DELETE	`/api/credentials/{id}`	Revoke a credential (soft delete)

c) *Administrative Endpoints:*

Endpoints for admin endpoints are restricted to only users who have the `IsRbacAdmin` flag, seen in the `User` table in Figure 2. This field is populated from the `ClaimsPrincipal` when provisioning the user in the `/api/me` route. Thus, it requires the OIDC Provider to include this claim on users who should have admin access. Alternatively, it can be set manually.

TABLE III
ADMIN ROLE MANAGEMENT ENDPOINTS PROVIDED BY OSWS

Method	Path	Description
GET	`/api/admin/columns`	Get all columns
POST	`/api/admin/columns/{columnId}/roles/{roleId}`	Grant access on column with ID `columnId`to role with id `roleId`
DELETE	`/api/admin/columns/{columnId}/roles/{roleId}`	Revoke access on column with ID `columnId` to role with ID `roleId`
GET	`/api/admin/roles`	Get all roles
POST	`/api/admin/roles`	Create a role
DELETE	`/api/admin/roles/{id}`	Delete role with ID `id`
POST	`/api/admin/roles/{parentId}/inherit/{childId}`	Make role with ID `parentId` inherit role with ID `childId`
DELETE	`/api/admin/roles/{parentId}/inherit/{childId}`	Delete inheritance of role `parentId` from role `childId`
GET	`/api/admin/users`	Get all users
POST	`/api/admin/users/{userId}/roles/{roleId}`	Grant role with ID `roleId` to user with ID `userId`
DELETE	`/api/admin/users/{userId}/roles/{roleId}`	Revoke role with ID `roleId` from user with ID `userId`

I. Frontend Architecture

A frontend for interacting with the endpoints described in Table II and Table III was built using Typescript-React. To quickly create a user-friendly working prototype, UI components from the open-source project `shadcn` [28] was used. A user can login using PocketID, as described in Section IV.D.b. Here, the user can create credentials to be used for the S3-Compatible API endpoint. If the user is an RBAC Admin, they get access to the admin panel for managing roles. The admin panel includes a table view for viewing currently existing users, roles, columns and permissions. To interact with the endpoints described in Table III, a "query editor" was built to manage RBAC operations in an SQL-like syntax, reminiscent of the syntax used to manage permissions in e.g. PostgreSQL.[6] Using the JavaScript library `peggyjs` [29], a simple grammar was written to parse statements into API calls. Some example statements to configure RBAC permissions and the API calls they parse to can be seen in Listing 2.

```
CREATE ROLE admin;
=> POST /api/admin/roles { name:
"admin" }
CREATE ROLE intern;
=> POST /api/admin/roles { name:
"intern" }
GRANT intern TO ROLE admin;
=> POST /api/admin/roles/1/inherit/2
GRANT ACCESS ON name TO intern;
=> POST /api/admin/columns/1/roles/1
GRANT ACCESS ON ssn TO admin;
=> POST /api/admin/columns/2/roles/2
GRANT admin TO USER alice;
=> POST /api/admin/users/1/roles/1
```

Listing 2. Example statements and their parsed API calls

V. METHODOLOGY

As the intention of OSWS was for it to work on top of S3 and with existing "fully managed all-in-one cloud platforms" and other query engines, both benchmarking and e2e tests are quite important. This section will cover both the setup of the benchmark and the setup of the e2e test.

A. Benchmarking

All benchmarking of OSWS was run on a Digital Ocean VM with access to 8 cores and 16GB of DRAM, see Appendix 1.b, together with Digital Ocean's own Object Store (Spaces), which is S3-compatible.

a) *E2E Benchmarks:*

As OSWS inherently implements different design choices compared to vanilla S3, no standardized benchmarking has been chosen, such as Warp which is otherwise used by multiple other Fortune 500 companies.[30], [31] The reasoning for this is that OSWS implements its own caching layer and uses a KV; as such, the benchmark would have to ensure that it tests both the throughput of N instances of OSWS running and that the configuration of caching and encryption is considered

[6] https://www.postgresql.org/docs/current/sql-grant.html

TABLE IV
MAIN CONFIGURATIONS OF BENCHMARKS THAT ARE TO BE RUN

Category	Benchmark	Configuration	Measurements
E2E	S3 (R2)	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
E2E	OSWS +encryption +caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
E2E	OSWS +encryption -caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
E2E	OSWS -encryption -caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
Micro	Permission Service	No. flat hierarchy roles: 4, 64, 256	Latency
Micro	Permission Hierarchy	No. hierarchical depth roles: 0, 4, 16, 64	Latency
Micro	Unwrapping	Key size: 128,192, 256 bits	Latency
Micro	Decryption	No. rows: 1,000, 10,000, 250,000, 1,000,000, 2,000,000 (50 columns)	Latency

thoroughly. OSWS also encrypts each column, resulting in the sizes of the Parquet files having a lot of significance for the end throughput, and Warp does not vary its Parquet file size. This results in inherently different throughputs and numbers, which are hard to compare OSWS against anything else. Lastly, OSWS, as mentioned, does not support proper range request, and it is then apparent that S3 will perform even better for ranged request, where regardless of the range or full file request, S3 will perform close to the same regardless.[7] Thus, the choice was to set up the following e2e benchmarks. The five Parquet files will be auto-generated by a script. These will have the following sizes:

- *Parquet Size Tiny:*~500KB Parquet file with 50 columns and 1,000 rows.
- *Parquet Size Small:*~5MB Parquet file with 50 columns and 10,000 rows.
- *Parquet Size Medium:*~125MB Parquet file with 50 columns and 250,000 rows.
- *Parquet Size Large:*~500MB Parquet file with 50 columns and 1,000,000 rows.
- *Parquet Size X-Large:*~1GB Parquet file with 50 columns and 2,000,000 rows.

This configuration was chosen to reflect a somewhat realistic distribution of different sizes of Parquet files. Various openly available datasets were compared to settle on the choice of 50 columns. For example, several real-world example datasets available from ClickHouse contain much fewer than 50 columns [32]. Another is a dataset collecting 1.1 billion taxi and Uber rides in New York City, which contains 51 columns [33]. 50 columns then seemed like a realistic amount of columns for a large OLAP dataset.

Python scripts will then run Parquet puts and gets against the following setups:

- Directly against an S3-compatible store
- Against OSWS with encryption and with caching
- Against OSWS with encryption and no file cache
- Against OSWS with encryption and no DEK cache
- Against OSWS with no encryption and no caching

The metric for this will be latency, and for each file, the benchmark will be run for a total of 10 times, both cold and warm, allowing comparison of how slow or speed differences with and without caches.

DigitalOcean [34] was chosen as hosting for both the benchmark driver through their Droplet virtual machine hosting, and S3-compatible storage through their Spaces Object Storage.

b) *Micro Benchmarks:*

The e2e benchmarks tell the overall story of the performance of OSWS, but it is essential to identify which parts within OSWS contribute to latency. The following micro-benchmarks will be run:

1) Permission Service
2) Permission Hierarchy
3) Unwrapping
4) Decryption

Permission Service will measure the latency of how long it takes to get a user's effective roles to the given columns. The columns stay at a constant 100, the hierarchy (meaning roles pointing to roles) stays at zero, the roles assigned to the user are varied from four, 64, and 256 roles, and they are each assigned, on average, access to 80% of each of the columns.

Permission Hierarchy will measure the latency of how long it takes to get a user's effective roles, but instead, roles are now linked to other roles and lastly to columns. The number of roles assigned is now constant at one, while the number of roles assigned in depth is varied from 0, 4, 16, and 64 and again, on average, each role is assigned access to 80% of the columns.

Unwrapping will measure the end-to-end latency for reading a tiny Parquet file (the small Parquet files are also run but ignored, but can be seen in Appendix 1.c) with a cold cache. The varying key sizes of 128, 192, and 256 bits will primarily affect the symmetric AES-GCM decryption.

Decryption will measure the latency for reading and decrypting a Parquet file, and the keys for decrypting will have been warmed and thus stored in cache; thus, KV is not taken into account here. The Parquet sizes that will be used are the same as with e2e benchmarking, being the tiny, small, medium, large, and extra large.

All of the micro-benchmarks were run 15 times to get a more consistent result.

See Table IV for an overview of the benchmarks and their configurations, which will be run.

[7] A solution to this is proposed within Section VII.

c) *Testing:*

As one of the goals of the OSWS was to enable third-party query engines to connect without any modification, a test confirming this behaviour is needed.

An end-to-end test will be created that asserts different query engines can connect and perform queries as normal, with OSWS providing column-level security based on the test user's roles. The end-to-end test suite will contain a seeding step that sets up the necessary users and roles. Python scripts will be used to orchestrate setting up roles and permissions, and asserting that the fetched data does not contain unauthorized data. The test will use the "Titanic" sample dataset[8] and permissions will be configured so two roles have limited access, but to different columns. One role will inherit both and have direct access to the rest of the columns, to thus have full access. This will then test the role hierarchy feature. Then, the following will be used to fetch the Parquet file through different users and assert that only the permitted columns are viewable:

Python script A python script using an S3 client like `boto3` and a Parquet reader like `pyarrow` to fetch a file from OSWS.

DuckDB A DuckDB instance will query the parquet file through OSWS.

Apache Spark An Apache Spark instance (through PySpark) will query the parquet file through OSWS.

Each of the different tools should assert that only the permitted data is viewable.

VI. Results

This section will cover the results of running both the benchmarks and e2e tests described in Section V, and quickly define what they mean.

A. E2E Tests

An end-to-end test suite was written to test Python using PyArrow, DuckDB and PySpark against OSWS as an S3 endpoint. No modifications were made to the tools outside of changing the S3-endpoint to an instance of OSWS. The test suite shows that all three tools are successfully able to query a Parquet file and perform operations on the result. The users are identified through their credentials, and columns the users roles do not give access to are masked correctly.

This shows that OSWS works as a drop-in replacement for S3 when working with query engines that fetch Parquet files directly and do not store metadata.

WIP - We are not sure what the best way to present the "results" of the (functioning) test is - it works? How do we show that in a report? Also, we will find a way to add the terminal output cleanly for good measure.

B. Benchmarking

a) *E2E Benchmarks:*

To begin with, comparison of cold GET latency with and without the DEK cache enabled is shown in Figure 3. Only tiny to medium is included due to the benchmark timing out on larger files (>600s). However, even without large, it is clear that the DEK cache is invaluable even on small files, and on larger files the system gets unusably slow without the DEK cache. At first it seemed surprising that the difference between a *cold* GET, that is, where the DEK cache is empty from the start, and a GET where the DEK cache is not enabled, is this large. However, it makes sense given that the DEK cache is warmed up as the file is being read, since reading a row chunk will populate the DEK cache with the columns of that chunk. Since the whole column is encrypted with the same DEK, once that column is reached again in a new row chunk, the cache is warm. This shows that the DEK cache is a must-have for OSWS. For that reason, the "no DEK cache" config is omitted from here on.

Figure 4 shows the results of rest of the end-to-end benchmark suite. Originally, the intent was to have an xlarge file size as well, but this turned out to be too slow to reasonably include in the plots. This also shows that there is a weakness when the files get to the 1GB size.

However, looking at Fig.4a for a **warm** GET it is again clear that the DEK cache is a must have. On smaller files, all configurations of OSWS actually come close to the direct DigitalOcean fetch, though there is clearly a lot of overhead still. Interestingly, the "no encrypt" is seemingly a bit *slower* than the other configurations for the "tiny" and "small" configuration, even though it should just be the pure network overhead. However, disabling encryption also disables the file cache, which might be why it is slower – due to it having to go to DigitalOcean every time.

In Fig 4b. the **cold** GET is shown. Direct fetch is omitted here due to there being no "cold" path – this is the same as Fig 4a. Here, the "no encryption" configuration is an order of magnitude faster for the "tiny" and "small" config, which highlights the overhead of the decryption step. However, as the files get larger the network overhead starts to show aswell, when the "no encryption" config starts to approach the other configurations.

Generally, both Fig 4a and 4b also show that the file cache is mostly useful when files are large; this is not surprising as the encrypted file cache only helps with reducing S3-fetches, as the file must still be decrypted again. But when the network cost starts getting visible, the file cache matters.

Fig. 4c shows the PUT latency. Unsurprisingly, there is massive overhead on the "tiny" configuration, as the direct upload is in millisecond-level, while OSWS must spend time encrypting the file and making calls to KV. Again, as files get larger the relative overhead reduces as network transfer becomes a factor.

b) *Micro-Benchmarks:*

In Figure 5, all box-plots increase in duration whenever their role depth, flat roles, or the number of rows increases (Note that the y axis is not equal, and some of them are linear and others logarithmic). The latency for Permission Hierarchy Benchmark, see Figure 5c, and Permission Service Benchmark, see Figure 5d, both have a minor contribution to latency. The major contributors to latency are decryption, see Figure 5a, and DEK unwrapping. The unwrapping of the DEKs remains close to constant. The DEK unwrapping is not

[8] https://www.agentsfordata.com/app?sample-id=titanic&tab=local

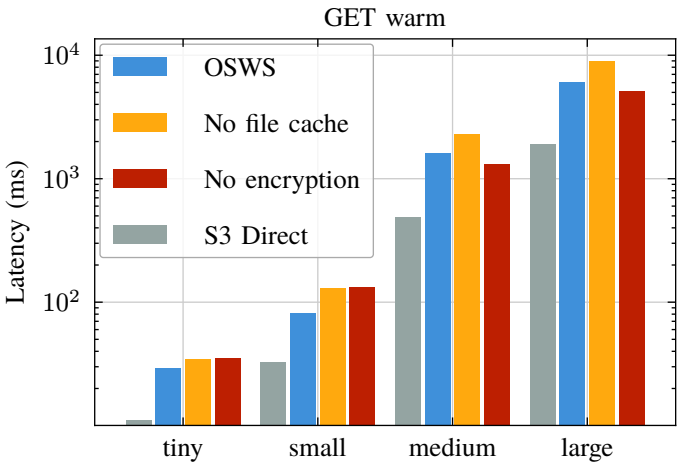

Fig. 4a. Median GET latency on warm cache

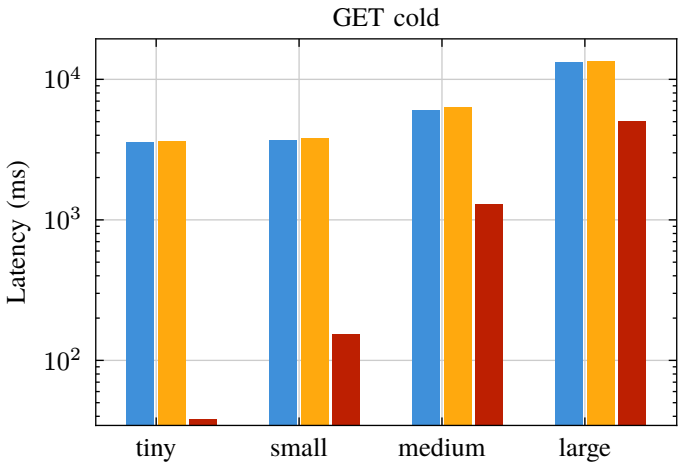

Fig. 4b. Median GET latency on cold cache

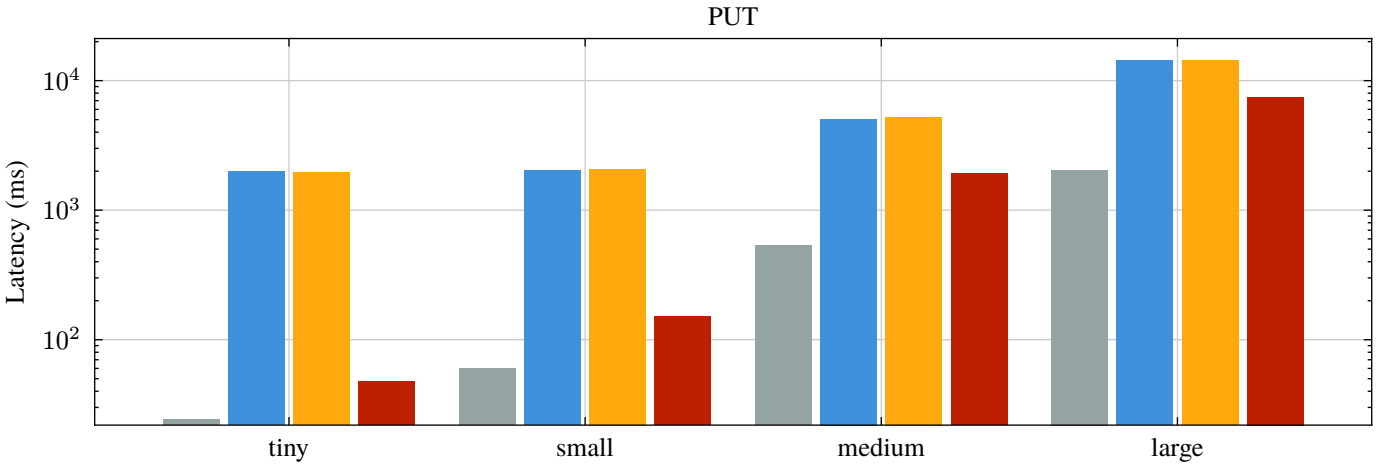

Fig. 4c. Median PUT latency

Figure 4. Plots showing median end-to-end latency measured during benchmarks. Grouped by file size. Note log scale on y-axis. Note that legend is shared.

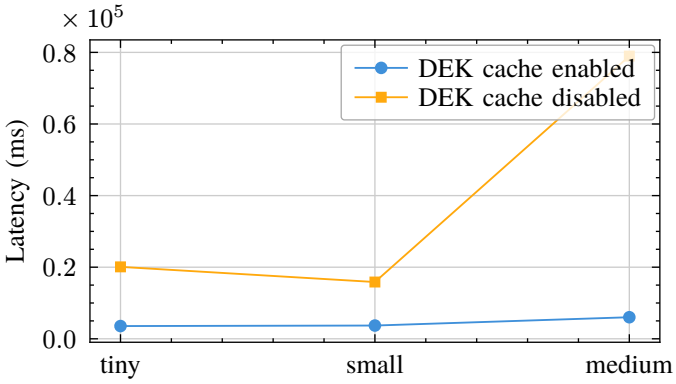

Figure 3. DEK cache impact on cold GET latency

something that can be improved much besides changing the KV service provider to a faster alternative, and then, as OSWS currently does, cache the unwrapped DEK heavily to use KV as little as possible. Decryption is easier to optimize, due to OSWS, as per version 2026-alpha, relying on Parquet Sharp version 21.0.0. To improve it, OSWS would need to implement a custom low-level decrypter, which would also allow for not needing to copy the partially decrypted Parquet file over to another Parquet file, but instead rewrite the columns in place, if decryption is needed. This would help, as the current decryption micro-benchmark also takes the copying into account, though the main improvement would be seen when columns are not authorized to the client, and can thus be left as is. So instead of the computations being directly related to the number of Parquet columns, they will be related to the number of authorized columns.

$$\text{Decryption: } O(|\text{columns}|) \geq O(|\text{columns}_{\text{authorized}}|) \quad (1)$$

VII. Discussion

OSWS set out to provide columnar access control to 3rd party query engines and "fully managed all-in-one cloud platforms" by encrypting columns individually using PME, provided through the Parquet Sharp library, see Section IV.E.b. In short, during the encryption flow, the Parquet size is changed due to how Parquet Sharp operates, as well as added metadata. This eliminates query engines, which use the range modifier based on cached sizes and do not read from the modified metadata. This then eliminates DuckLake from using OSWS, as it caches the inserted ranges [35]. The same issue is prevalent for "fully managed all-in-one cloud platforms", due to the incompatible design choices made [36], [37].

OSWS was intended to support all 3rd party query engines and "fully managed all-in-one cloud platforms", but in its

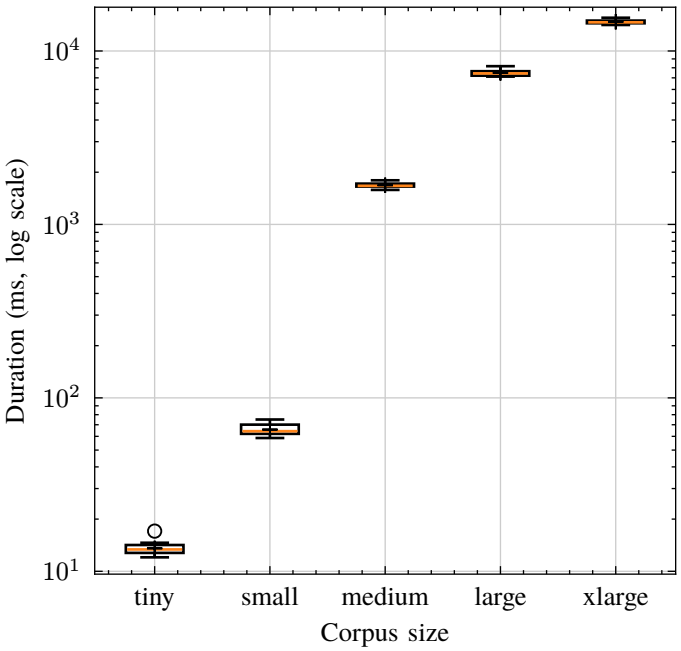

Fig. 5a. Decryption boxplot distribution per parameter (logarithmic y axis)

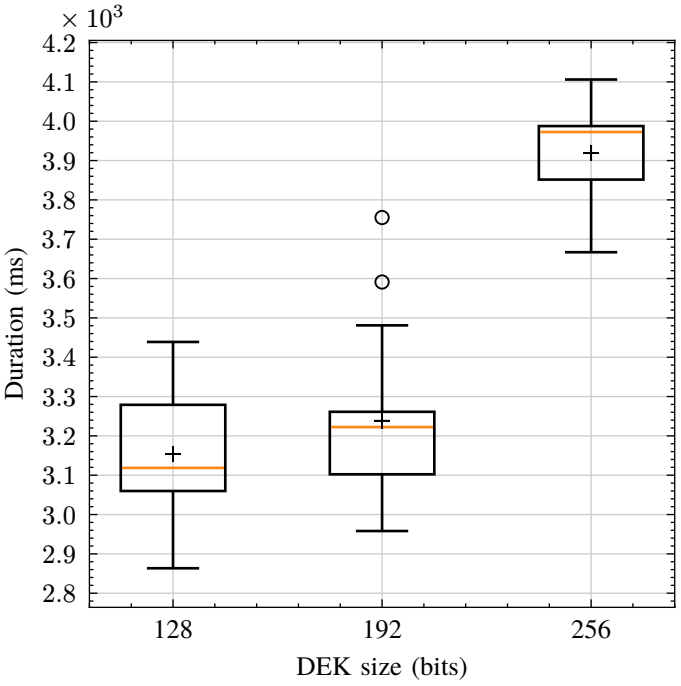

Fig. 5b. KeyUnwrap box-plot distribution per parameter set (tiny corpus only; linear y axis)

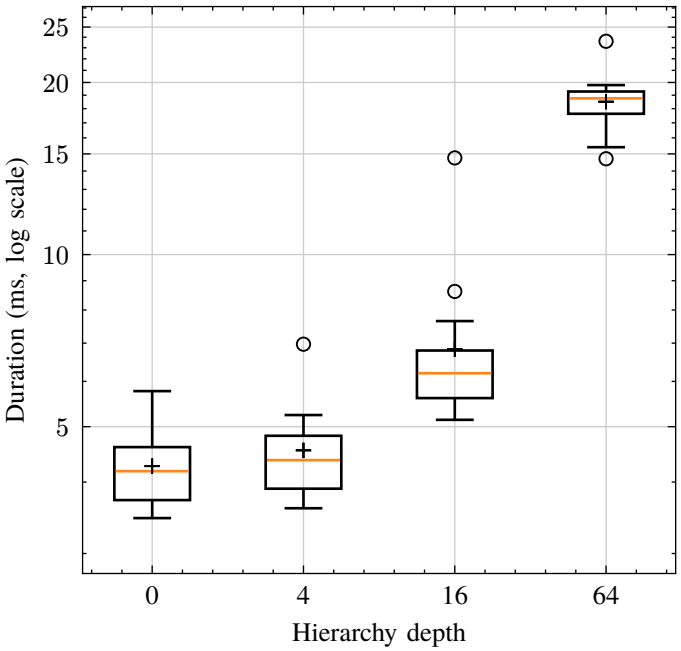

Fig. 5c. PermissionHierarchy box-plot distribution per parameter set (logarithmic y axis)

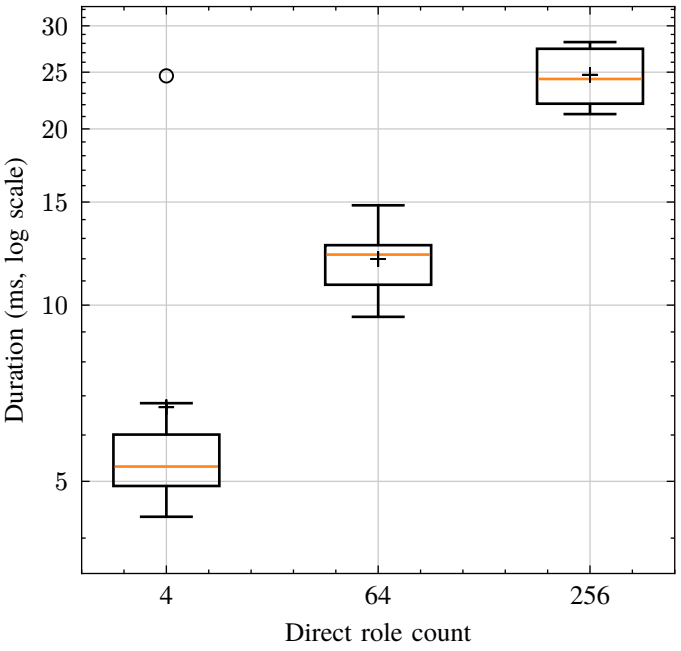

Fig. 5d. PermissionService box-plot distribution per parameter set (logarithmic y axis)

Figure 5. Box-plots for micro-benchmarks. Note that the y axes are not the same and some of the plots are logarithmic

current state the platforms and some query engines are not supported. For the platforms, it has not been possible to test OSWS, as it would need the providers to whitelist a URL where OSWS would run. But most of these incompatibility issues are from the initial design choices made during the start, but first became apparent on the final iteration of designing benchmarking and e2e tests, which, based on the research paper [1], made sense, but after modifying it to work with the various providers, did not make sense anymore and had unnecessary overhead or created other problems, and as such this section will address those issues, define what should have been done instead, and define which performance done in OSWS which can be removed.

A. *Metadata*

The idea for OSWS was to allow third-party query engines to decrypt Parquet files themselves locally, while decrypting within OSWS for “fully-managed cloud data lake platforms”.

This would be achieved by storing key IDs within the footer of the Parquet files, and the query engines could then, through an endpoint in OSWS, request the given key and then decrypt the column locally, and OSWS would do the logic itself. As soon as it was identified that this was not possible with KV to retrieve the keys from it, OSWS was changed to use envelope encryption and store the wrapped DEKs, and the KEK reference in the footer, due to it being partly meant for that purpose. When a client puts a file through OSWS, the Parquet file size is modified to contain related wrapped DEKs and KEK reference. This breaks many query engines, where they themselves store size metadata, and range requests are not fully functional. The solution to this would most likely be to have an internal SQL server running in OSWS. This database should contain the following information per Parquet file inserted:

1) Internal File Identifier for footer[9]
2) Byte range of the start and end index
3) Internal File Identifier for Column Index[9]
4) Wrapped DEK
5) KEK reference
6) Row chunk size

Together with a full range of requests, support for more concurrency can be used. When OSWS get a request to read a file, it can start retrieving it. In the meantime, it can also go to the local database and find files which match *1*. It can then see which keys the client has access to in the RBAC database, and then match the columns with *3*, then for each send *4* to KV given *5*.

Another improvement is when range requests are given, OSWS can figure out which ranges are related to which columns given *2* and then retrieve it from S3. Meanwhile it can retrieve the unwrapped DEKs, to be ready for decryption. Now for decryption it can use the row chunk size *6* to be able to decrypt the small part, though only possible if AES CTR is used, more on this in Section VII.B.

So for metadata, it should only be saved within internal SQL, still, as the current solution relies on SQL, and then the encryption of the columns also needs to change.

B. Encryption & Decryption

Currently, encryption and decryption are handled by Parquet Sharp, which is a .NET package which supports PME. Choosing this library was a mistake, as it handles encryption and decryption by reading everything (using cryptographic keys if provided) and copying it into a new Parquet file. If no cryptographic key is provided to an encrypted column, it fails. So here are two major issues with how it handles cryptography:

1) It reads and copies the entire file regardless of whether a single column has to be decrypted or all of them.
2) It does not allow for copying over encrypted columns.

Also, as Section VII.A, the size of the Parquet file changes upon encryption, due to PME, and thus creates issues for range requests.

All this could be solved by using the implementing a custom reader and writer for the Parquet files, which would use decrypt and encrypt in place, thus fixing *1*, being able to leave non-authorized columns encrypted, thus solving *2*, and not changing metadata and using the solution described in Section VII.A. Important for the encryption is that AES CTR, which does not use any padding, and thus is also a contributor to not modifying any part of the size of the file.Dworkin_2001 The issues and things to account for when using CTR have been discussed by [39] and include the following:

- No integrity: CTR provides no message integrity, but can be handled by MAC.
- Error propagation: Bit flips are localized and not propagated. Though this issue should be solved in another layer.
- Stateful encryption: Important that the system does not reuse keys, but that is already how OSWS operates.
- Sensitivity to usage errors: Counter-values are not to be reused, but given that OSWS generates a new key for each column, this can be ignored.

To argue even more for why CTR is the correct choice: given a column size in the Parquet file, there are multiple row chunks, which individually are encrypted. By knowing the column chunk size and the column size, the OSWS could identify the offset within the range query a client might have sent, only fetching the range from S3 and only decrypting that part, essentially removing a big part of the overhead OSWS currently experiences.

a) *Parallelization:*

When encrypting/decrypting the Parquet file, OSWS currently copies each row group sequentially. This should be an "embarrassingly parallelisable" operation, i.e. it could spawn threads to copy a cutout of the row groups to the new Parquet file so the row groups are copied in parallel. This should reduce copying overhead drastically, though in light of Section VII.B, it should be a write in place instead of a copy.

C. Compression

For Parquet Sharp to copy over a column, it has to decompress the file. Here, OSWS thus modifies the file size and just keeps the file decompressed afterwards, when put into S3. It could have compressed it again, but it would not have fixed range queries due to modified metadata, etc. Here, with the custom reader and writer, it is not even needed to decompress the file. OSWS could potentially just keep the current format and still encrypt, etc., on a compressed or non-compressed file and thus leave all the sizes untouched.

D. Encrypted Parquet Storage Cache

When Parquet files are fetched from S3, they are stored in local storage in their encrypted format. But as seen in Section VI.B.a, it has negligible performance improvements on the smaller files, while more significant improvements on larger Parquet files. This improvement might even become smaller and less useful with proper range support, and will thus be one of the later improvements which might be tested

[9]Defined within [38]

on a future OSWS system. Of course, putting the caching in their unencrypted state and in DRAM cache could enhance performance as well, but would raise issues with handling access control on columns and size issues of the Parquet files, ending up filling up DRAM too quickly.

E. Correct Decisions & OSWS Future

Even though OSWS has a lot of design choices, in light of prior subsections, a lot of correct decisions are believed to have been implemented, and are believed to need to be carried over to a V2 of a similar system.

a) *Envelope Encryption:*

Envelope encryption should still be used, even though it carries an initial significant overhead, but it is partly solved by having the decrypted DEK cache. This will also enable easier key rotation for future security improvements. A potential alternative would be to implement a fully fledged key vault inside OSWS, eliminating envelope encryption, though this would require many security measures.

b) *DEK Cache:*

The decrypted DEK cache is another one of these design choices which has to be carried over to a new version, given that a KV is used, as seen by Section VI.B.a and Section VI.B.b, that unwrapping DEKs carries a significant overhead.

c) *RBAC Metadata Storage:*

The microbenchmarks in Section VI.B.b also showed that the impact of authorizing users using the RBAC database was insignificant compared to the overhead of cryptographic operations and network transfer in general. This shows that a PostgreSQL database with the implemented design is a good option for RBAC Metadata Storage in the future.

d) *Role Management:*

Role management (creating and assigning roles, granting permissions) using the implemented "query editor" provided an intuitive and familiar way of managing roles. Though this is just a wrapper over the admin API, and it could be implemented in many ways, this "query editor" is a valid option that could be used in the future.

VIII. Conclusion

The current implementation of OSWS proves that fine-grained role-based access control is possible within data lakes, and that query engines which do not cache sizing data can use it without any modifications, and that, depending on whether the "fully managed all-in-one cloud platforms" use it or not, the only change needed is to whitelist a third-party link to OSWS.

A. Future Work

As suggested, it is believed that OSWS in its current state is not the proper implementation of how such a system should work, but that it could have a place within the current ecosystem of data lake uses.

The improvements needed are to implement a custom reader and writer, together

One of the most important changes is to implement the custom reader and writer as mentioned in Section VII.B.

I. Appendix

A. *Encrypted Parquet Footer*

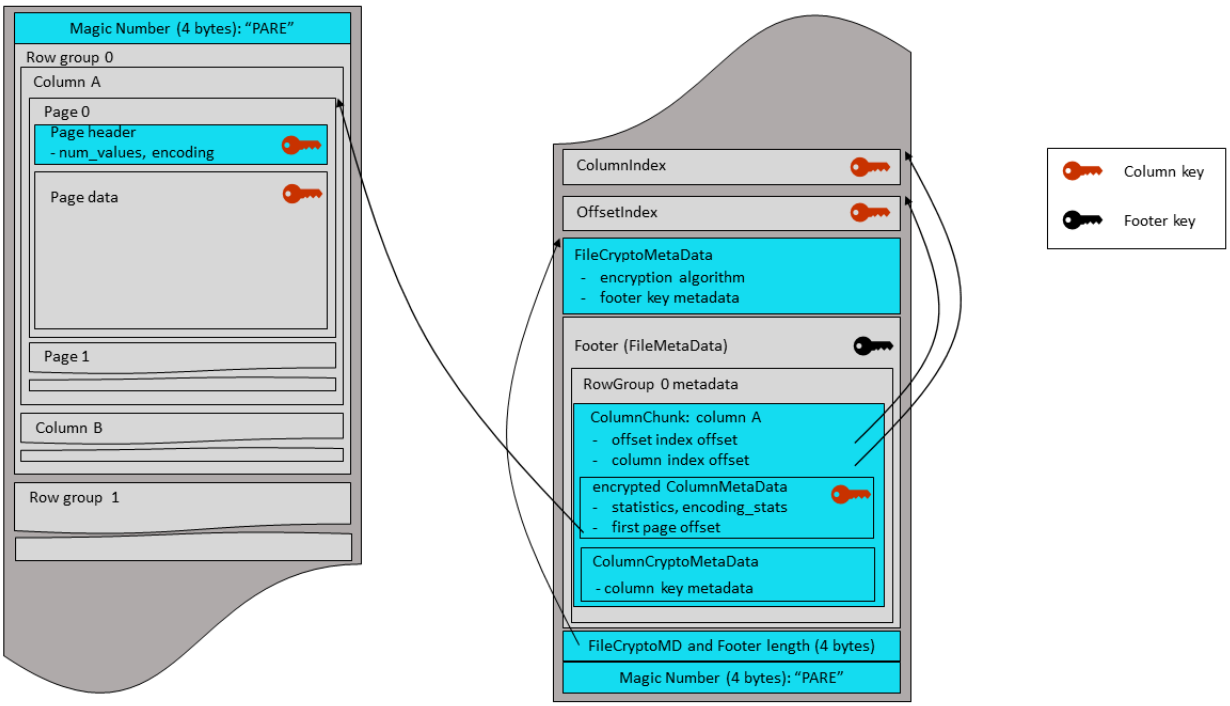

Fig. 1. Illustration of the Parquet file structure using an Encrypted Footer. From [17].

B. VM Specification

```
root@ubuntu-s-1vcpu-1gb-35gb-intel-fra1:~# inxi -Fxz
System:
  Kernel: 6.8.0-111-generic arch: x86_64 bits: 64 compiler: gcc v: 13.3.0
  Console: pty pts/6 Distro: Ubuntu 24.04.4 LTS (Noble Numbat)
Machine:
  Type: Kvm Mobo: DigitalOcean model: Droplet v: 20171212 serial: <filter> BIOS:
DigitalOcean
    v: 20171212 date: 12/12/2017
CPU:
  Info: 8-core model: DO-Premium-AMD bits: 64 type: MCP arch: Zen 2 rev: 0 cache: L1:
512 KiB
    L2: 4 MiB L3: 16 MiB
  Speed (MHz): avg: 1996 min/max: N/A cores: 1: 1996 2: 1996 3: 1996 4: 1996 5: 1996
6: 1996
    7: 1996 8: 1996 bogomips: 31940
  Flags: avx avx2 ht lm nx pae sse sse2 sse3 sse4_1 sse4_2 sse4a ssse3 svm
Graphics:
  Device-1: Red Hat Virtio 1.0 GPU driver: virtio-pci v: 1 bus-ID: 00:02.0
  Display: server: No display server data found. Headless machine? tty: 110x29
    resolution: 1024x768
  API: EGL v: 1.5 drivers: kms_swrast,swrast platforms: active:
gbm,surfaceless,device
    inactive: wayland,x11
  API: OpenGL v: 4.5 vendor: mesa v: 25.2.8-0ubuntu0.24.04.1 note: console (EGL
sourced)
    renderer: llvmpipe (LLVM 20.1.2 256 bits)
Audio:
  Message: No device data found.
  API: ALSA v: k6.8.0-111-generic status: inactive
Network:
  Device-1: Intel 82371AB/EB/MB PIIX4 ACPI vendor: Red Hat Qemu virtual machine
    type: network bridge driver: N/A port: N/A bus-ID: 00:01.3
  Device-2: Red Hat Virtio network driver: virtio-pci v: 1 port: c1a0 bus-ID: 00:03.0
  IF: eth0 state: up speed: -1 duplex: unknown mac: <filter>
  Device-3: Red Hat Virtio network driver: virtio-pci v: 1 port: c1c0 bus-ID: 00:04.0
  IF: eth1 state: up speed: -1 duplex: unknown mac: <filter>
  IF-ID-1: br-cf3f443f079d state: down mac: <filter>
  IF-ID-2: docker0 state: down mac: <filter>
Drives:
  Local Storage: total: 320 GiB used: 19.4 GiB (6.1%)
  ID-1: /dev/vda model: N/A size: 320 GiB
  ID-2: /dev/vdb model: N/A size: 488 KiB
Partition:
  ID-1: / size: 308.93 GiB used: 19.28 GiB (6.2%) fs: ext4 dev: /dev/vda1
  ID-2: /boot size: 880.4 MiB used: 113.9 MiB (12.9%) fs: ext4 dev: /dev/vda16
  ID-3: /boot/efi size: 104.3 MiB used: 6.1 MiB (5.9%) fs: vfat dev: /dev/vda15
Swap:
  Alert: No swap data was found.
Sensors:
  Src: lm-sensors+/sys Message: No sensor data found using /sys/class/hwmon or lm-
sensors.
Info:
  Memory: total: 16 GiB available: 15.62 GiB used: 1.03 GiB (6.6%)
  Processes: 180 Uptime: 4d 1h 2m Init: systemd target: graphical (5)
  Packages: 813 Compilers: N/A Shell: Bash v: 5.2.21 inxi: 3.3.34
```

Listing 3. VM specification used for the benchmarks of OSWS

C. Micro Benchmark Results

```
Benchmark: Decryption

parameters                    n         mean       stddev          min          p50
p95          p99          max
-------------------------------------------------------------------------------------------------------------------------------
  size=large                 15        7.48s      313.4ms        7.12s        7.43s
8.17s        8.17s        8.17s
  size=medium                15        1.68s       64.1ms        1.58s        1.67s
1.80s        1.80s        1.80s
  size=small                 15       65.7ms        4.8ms       58.7ms       64.2ms
75.0ms       75.0ms       75.0ms
  size=tiny                  15       13.6ms        1.2ms       12.0ms       13.4ms
17.1ms       17.1ms       17.1ms
  size=xlarge                15       14.7s       452.1ms       14.1s        14.6s
15.5s        15.5s        15.5s


Benchmark: KeyUnwrap

parameters                    n         mean       stddev          min          p50
p95          p99          max
-------------------------------------------------------------------------------------------------------------------------------
  size=small,dek_bits=128    15        4.15s      166.5ms        3.94s        4.15s
4.55s        4.55s        4.55s
  size=small,dek_bits=192    15        4.05s      198.0ms        3.76s        3.99s
4.37s        4.37s        4.37s
  size=small,dek_bits=256    15        4.05s      186.2ms        3.73s        4.04s
4.53s        4.53s        4.53s
  size=tiny,dek_bits=128     15        3.15s      152.0ms        2.86s        3.12s
3.44s        3.44s        3.44s
  size=tiny,dek_bits=192     15        3.24s      214.5ms        2.96s        3.22s
3.76s        3.76s        3.76s
  size=tiny,dek_bits=256     15        3.92s      113.8ms        3.67s        3.97s
4.11s        4.11s        4.11s


Benchmark: PermissionHierarchy

parameters                    n         mean       stddev          min          p50
p95          p99          max
-------------------------------------------------------------------------------------------------------------------------------
  hierarchy_depth=0          15        4.3ms        0.7ms        3.5ms        4.2ms
5.8ms        5.8ms        5.8ms
  hierarchy_depth=16         15        6.8ms        2.3ms        5.1ms        6.2ms
14.8ms       14.8ms       14.8ms
  hierarchy_depth=4          15        4.5ms        0.8ms        3.6ms        4.4ms
7.0ms        7.0ms        7.0ms
  hierarchy_depth=64         15       18.5ms        2.0ms       14.7ms       18.8ms
23.6ms       23.6ms       23.6ms


Benchmark: PermissionService

parameters                    n         mean       stddev          min          p50
p95          p99          max
-------------------------------------------------------------------------------------------------------------------------------
  role_count=256             15       24.7ms        2.6ms       21.2ms       24.3ms
28.1ms       28.1ms       28.1ms
  role_count=4               15        6.7ms        4.8ms        4.3ms        5.3ms
24.6ms       24.6ms       24.6ms
  role_count=64              15       12.0ms        1.4ms        9.6ms       12.2ms
14.8ms       14.8ms       14.8ms
```

Listing 4. Textual representation for the results of running the micro benchmarks

References

[1] A. S. H. Trøstrup and L. F. T. Hanson, "Reviewing options for fine-grained Role-Based Access Control in Data Lakes." [Online]. Available: https://github.com/lucasfth/reviewing-options-for-fine-grained-role-based-access-control-in-data-lakes

[2] S. Kumar, S. Yagati, C. Power, D. E. Culler, and R. A. Popa, "Membrane: A Cryptographic Access Control System for Data Lakes," *ArXiv*, 2025, [Online]. Available: https://api.semanticscholar.org/CorpusID:281243750

[3] A. Jonker and M. M. Kosinski, "What is a data lake?." [Online]. Available: https://www.ibm.com/think/topics/data-lake

[4] Deepstrike, [Online]. Available: https://deepstrike.io/blog/what-is-jit-provisioning

[5] D. F. Ferraiolo and R. Kuhn, "Role-Based Access Controls," in *Proceedings of the 15th National Computer Security Conference (NCSC)*, Baltimore, MD, USA, Oct. 1992, pp. 554–563.

[6] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001, doi: 10.1145/501978.501980.

[7] O. Foundation, "What is OpenID Connect." [Online]. Available: https://openid.net/developers/how-connect-works/

[8] Microsoft, "What is Microsoft Entra?." [Online]. Available: https://learn.microsoft.com/da-dk/entra/fundamentals/what-is-entra

[9] PocketID, "Introduction to PocketID." [Online]. Available: https://pocket-id.org/docs/introduction

[10] AWS, "AWS Key Management Service." [Online]. Available: https://docs.aws.amazon.com/kms/latest/developerguide/overview.html

[11] Azure, "Azure Key Vault Overview - Azure Key Vault." [Online]. Available: https://docs.azure.cn/en-us/key-vault/general/overview

[12] AWS, [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

[13] A. S. Foundation, "Apache Spark." [Online]. Available: https://spark.apache.org/

[14] M. A. Zaharia, A. Ghodsi, R. Xin, and M. Armbrust, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *Conference on Innovative Data Systems Research*, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:229576171

[15] Apache, "Parquet File Format." [Online]. Available: https://parquet.apache.org/docs/file-format/

[16] A. S. Foundation, "Apache Thrift." [Online]. Available: https://thrift.apache.org/

[17] Apache, [Online]. Available: https://parquet.apache.org/docs/file-format/data-pages/encryption/

[18] Redis, "TTL." [Online]. Available: https://redis.io/docs/latest/commands/ttl/

[19] E. Barker, *Recommendation for Key Management Part 1: General*, no. NIST SP 800-57pt1r4. 2016, p. NIST SP 800–57pt1r4. doi: 10.6028/NIST.SP.800-57pt1r4.

[20] Google, [Online]. Available: https://docs.cloud.google.com/kms/docs/envelope-encryption

[21] S. Myagmar, A. J. Lee, and W. Yurcik, "Threat Modeling as a Basis for Security Requirements," 2005.

[22] Lakekeeper, "Lakekeeper Docs - Storage." [Online]. Available: https://docs.lakekeeper.io/docs/0.5.x/storage/None

[23] Apache, [Online]. Available: https://polaris.apache.org/releases/1.4.1/

[24] R. Anderson and T. Dykstra, "Tutorial: Create a Minimal API with ASP.NET Core." [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/tutorials/min-web-api?view=aspnetcore-10.0

[25] "Authenticating Requests (AWS Signature Version 4)." [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/API/sig-v4-authenticating-requests.html

[26] [Online]. Available: https://bitwarden.com/resources/passkeys-are-phishing-resistant-mfa/

[27] msmbaldwin, "Azure data encryption at rest - Azure Security." [Online]. Available: https://learn.microsoft.com/en-us/azure/security/fundamentals/encryption-atrest

[28] shadcn, "Introduction to shadcn." [Online]. Available: https://ui.shadcn.com/docs

[29] peggyjs, "peggyjs/peggy – Parser Generator for JavaScript." [Online]. Available: https://github.com/peggyjs/peggy

[30] "Companies using MinIO in 2026 | Landbase." Accessed: Mar. 23, 2026. [Online]. Available: https://data.landbase.com/technology/minio/

[31] "Companies that use Minio (2,450)." Accessed: Mar. 23, 2026. [Online]. Available: https://theirstack.com/en/technology/minio

[32] ClickHouse, "ClickHouse Example Datasets." [Online]. Available: https://clickhouse.com/docs/getting-started/example-datasets

[33] T. W. Schneider, "Analyzing 1.1 Billion NYC Taxi and Uber Trips, with a Vengeance." [Online]. Available: https://toddwschneider.com/posts/analyzing-1-1-billion-nyc-taxi-and-uber-trips-with-a-vengeance/

[34] DigitalOcean, "DigitalOcean." [Online]. Available: https://www.digitalocean.com/

[35] DuckDB Foundation, "ducklake_data_file." [Online]. Available: https://ducklake.select/docs/stable/specification/tables/ducklake_data_file.html

[36] Snowflake, "Introduction to external tables." [Online]. Available: https://docs.snowflake.com/en/user-guide/tables-external-intro?utm_source=DV-Blog%3Fsource&utm_cta=website-workload-ai-ml-timely-content-building-a-chatbot-app-with-rag-and-feature-store

[37] Snowflake, "External table files." [Online]. Available: https://docs.snowflake.com/en/sql-reference/functions/external_table_files

[38] "Parquet Modular Encryption." [Online]. Available: https://parquet.apache.org/docs/file-format/data-pages/encryption/

[39] H. Lipmaa, P. Rogaway, and D. Wagner, "Comments to NIST concerning AES-modes of operations: CTR-mode encryption," p. , 2000.